

Futura Business Informatique

Sikkerhetsrevisjon 2024



HKSecurity

Dokument navn/versjon: 2024-10-12 FPI PEN-T ver 2.0 SBM

Futura Business Informatique – sikkerhetsrevisjon 2024 – Client Confidential

Contents

Executive Summary	4
Konfidensialitetsavtale	5
Versjonslogg	5
Forkortelser	5
Omfang	5
Begrensninger	5
Gjennomførelse	6
Deltagere	6
Verktøy	6
Tidslinje	6
Testtype	6
Ansvarsfraskrivelse	7
Oversikt over sårbarheter	8
Sårbarhetskake	8
Sårbarhetsvurdering	8
Sårbarhetstabell	9
Risikomatrikse	9
Detaljert beskrivelse av funn	9
Sårbarhet 1 - HTTP	9
Sårbarhet 2 - Clickjacking	11
Sårbarhet 3 - SQLI	13
Sårbarhet 4 – Apache versjon 2.4.38	14
Sårbarhet 5 – tilgang til admin.php	15
Sårbarhet 6 - XSS	16
Sårbarhet 7 – X-Content-Type-Options og MIME-sniffing	18
Sårbarhet 8 – Cookies uten httpOnly flag	18
Sårbarhet 9 – Cookies uten sameSite attribute	19
Sårbarhet 10 – databasefil ikke slettet	19
Sårbarhet 11 – md5 kryptering	20
Sårbarhet 12 – database passord	21
Informasjon 1 – README ikke blokkert	22
Informasjon 2 – Falsk positiv	22
Informasjon 3	22



Informasjon 4	22
Informasjon 5	23
Informasjon 6	23
Informasjon 7	23
Kilder	23

Executive Summary

1/3 side

Denne testen er utført av Sunniva Bakke Moan, Cybersikkerhetsstudent ved Kristiania i Bergen. Gjennom ulike tester osv fant vi ut at denne nettsiden har kritisk risiko for exploitation. Dette bør dere gjøre fremover... og disse bør jobbene utlyse...

Konfidensialitetsavtale

Dette er en sikkerhetsrevisjon gjennomført av Sunniva Bakke Moan I emnet etisk hacking/ETH2100. Dette dokumentet er konfidensielt og klientens eiendom og må ikke benyttes utover det som er avtalt med klienten i hovedkontraktsdokumentet FBI-453-654 Security Commercial Agreement.

Versjonslogg

Dokument navn: 2024-10-12 FBI PEN-T

Versjon	Dato	Endring	Utført av
1.0	29.10.2024	Første rapport	Sunniva B. Moan
2.0	13.10.2024	Utdypning og profesjonalisering	Sunniva B. Moan

Forkortelser

Forkortelser benyttet i dette dokumentet

PEN Test

FBI

Omfang

The boundaries and the needs of the engagement agreed upon with the organization.

Begrensninger

Limitations can be:

- Out-of-bounds areas in relation to testing.
- Broken functionality.
- Lack of cooperation.

- Lack of time.
- Lack of access or credentials.

Gjennomførelse

Testen retter seg mot følgende inngangspunkt:

- <http://192.168.23.132/>

Deltagere

Testen ble utført av Sunniva Bakke Moan, Cybersikkerhets-student på Kristiania.

Verktøy

Verktøy benyttet i denne analysen

Verktøy	Versjon	Kommentar
Lenovo Yoga PC		
Nikto		
OWSP ZAP		
Nessus		
Wireshark		
Kali Linux		
NMAP		
Vscode		

Tidslinje

Versjon 1.0 -

Versjon 2.0 -

Testtype

White box test

Ansvarsfraskrivelse

You may wish to provide a disclaimer for your service. Always consult a legal professional in order to create a legally-binding document.

The following example is for illustrative purposes only. It should not be used as-is and does not constitute legal advice.

This test is a “point in time” assessment and as such the environment could have changed since the test was run. There is no guarantee that all possible security issues have been identified, and that new vulnerabilities may have been discovered since the tests were run. As such, this report serves as a guiding document and not a warranty that the report provides a full representation of the risks threatening the systems at hand.

Oversikt over sårbarheter

Sårbarhetskake

SÅRBARHETS KAKE - må gjøres etter at alle sårbarhetene er plassert riktig

Sårbarhetsvurdering

Level	Score	Beskrivelse
Kritisk	10	The vulnerability poses an immediate threat to the organization. Successful exploitation may permanently affect the organization. Remediation should be immediately performed.
Høy	7-9	The vulnerability poses an urgent threat to the organization, and remediation should be prioritized.
Medium	4-6	Successful exploitation is possible and may result in notable disruption of business functionality. This vulnerability should be remediated when feasible.
Lav	1-3	The vulnerability poses a negligible/minimal threat to the organization. The presence of this vulnerability should be noted and remediated if possible.
Informasjon	0	These findings have no clear threat to the organization, but may cause business processes to function differently than desired or reveal sensitive information about the company.

Sårbarhetstabell

Kategori	Sårbarhet	Risiko
Sårbarhet 1	http brukes uten kryptering	Høy
Sårbarhet 2	Siden er sårbar for clickjacking	Høy
Sårbarhet 3	Siden er sårbar for SQLI MYE IGJEN	Høy
Sårbarhet 4	Apache versjonen som kjører er utdatert	Høy
Sårbarhet 5	Tilgang til /admin/admin.php ikke blokkert NESTEN FERDIG	Medium
Sårbarhet 6	Siden er sårbar for XSS angrep NESTEN FERDIG	Medium
Sårbarhet 7	Sårbar for MIME-sniffing ved manglende X-Content-Type-Options	Lav
Sårbarhet 8	Cookie uten httpOnly flag. (beholde men endres) IKKE FERDIG	Lav
Sårbarhet 9	Cookie uten SameSite Attribute. (?) IKKE FERDIG	Lav
Sårbarhet 10	Database opprettelse fil ikke slettet (medium?)	Lav
Sårbarhet 11	Passord er kryptert med hashing md5	Lav
Sårbarhet 12	Svakt passord på databasen	Lav
Informasjon 1	Apache /icons/README default fil er ikke blokkert	Info
Informasjon 2	Falsk positiv – «login.php: admin login page/section found»	Info
Informasjon 3	Sjekk nessus	Info
Informasjon 4	Sjekk nessus	Info

Risikomatrise

Vurderer sårbarhetene opp mot konsekvenser og reel sannsynlighet

Detaljert beskrivelse av funn

Sårbarhet 1 - HTTP

http brukes uten kryptering. http er en protokoll som ikke benytter seg av TLS eller SSL for å kryptere og sikre dataen. SSL og TLS er protokoller som krypterer data som sendes mellom en klient og en server på web sider. http regnes som en usikker protokoll og er ikke egnet for en bank sin nettside. Dette er fordi http sender data i klartekst uten å først kryptere det, i motsetning til det https gjør.

Oppdagelse

Nettsiden viser ingen indikasjon på bruk av kryptering av data og ved en skanning av portene med nmap på siden, ble port 80 som er portnummeret for http oppdaget.

Ved å åpne siden, ser man raskt at siden ikke er sikker, både i chrome og i firefox. I chrome fikk jeg opp et advarselstegn og «ikke sikker» foran og i firefox fikk jeg opp en sikkerhetsnøkkel med strek over seg foran ip-adressen. Dette indikerer at siden ikke er sikker og bruker http for å sende pakker.

I tillegg testet jeg dette i wireshark, for å se om jeg fikk opp trafikk i klartekst. Dette gjorde jeg ved å skrive inn TEST i brukernavnfeltet og TESTPASSORD i passordfeltet. Når jeg sjekket wireshark, fikk jeg opp både brukernavnet og passordet jeg hadde skrevet, i klartekst.

Konsekvenser

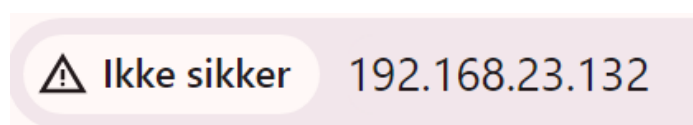
Her kan angriper for eksempel bruke wireshark som er en nettverkssniffer for å utføre man-in-the-middle angrep ved å lese dataen som blir sendt. Siden det ikke blir kryptert vil dataen vises i klartekst (som vist i wireshark under) og sensitiv informasjon slik som brukernavn, passord, transaksjoner, meldinger og annen konfidensiell informasjon som sendes, vil være synlig.

Ved bruk av http kan angriper også manipulere og endre dataen som sendes, slik at meldingene man tror man får direkte fra kilden, egentlig går gjennom angriper og endres før den kommer frem.

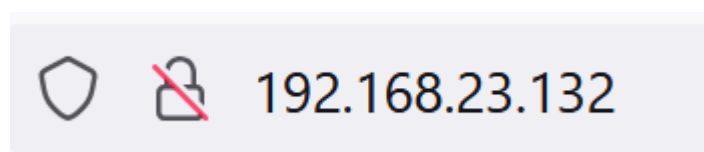
Løsning

Løsningen her er å implementere https ved å bruke TLS/SSL på serveren. Jeg anbefaler på det sterkeste å bruke TLS ettersom at det er en nyere, sikrere og forbedret versjon av SSL, som har mange kjente sikkerhetsfeil. Dette gjøres ved å skaffe et TLS sertifikat fra en CA (certificate authority), generere en CSR og deretter installere og konfigurere TLS-sertifikatfilene inn i konfigurasjonsfilen til nettsiden. Deretter ville jeg testet konfigen for å se om det er noe feil og så startet serveren på nytt. For en enda sikrere side kan man også legge til en HSTS som passer på at siden alltid bruker https.

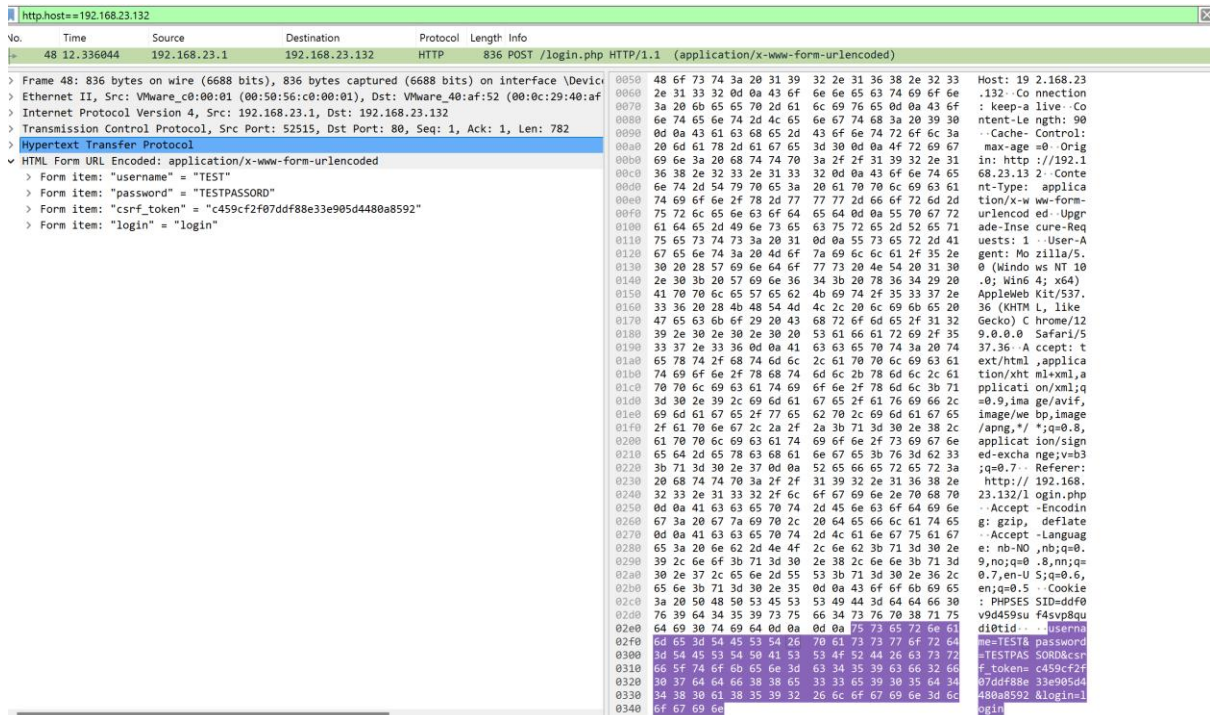
Dokumentasjon



Figur 1.1: Chrome



Figur 1.2: Firefox



```
HTML Form URL Encoded: application/x-www-form-urlencoded
> Form item: "username" = "TEST"
> Form item: "password" = "TESTPASSORD"
> Form item: "csrf_token" = "c459cf2f07ddf88e33e905d4480a8592"
> Form item: "login" = "login"
```

Figur 1.4: Test brukernavn og passord vist i klartekst i wireshark

Sårbarhet 2 - Clickjacking

X-Frame-Options header er ikke satt og siden er sårbar for clickjacking. En X-Frame-Options header begrenser hvorvidt en nettleser kan laste opp siden i frames eller objekter i scripts. Hvis en side kan bli lastet opp i et script kan en angriper skrive kode som får knapper på siden til å gjøre noe annet enn det de ser ut til å gjøre. Dette kalles clickjacking.

Oppdagelse

I en nikto scan av nettsidens Ip-adresse og en owasp zap active scan fikk jeg opp at anti-clickjacking headeren X-Frame-Options ikke var satt. Siden dette er en header som beskytter nettsider mot clickjacking, ville jeg teste ut om det faktisk gikk an å clickjacke siden.

I tillegg fikk jeg opp at CSP headeren heller ikke tas i bruk, i en owasp zap scan og en Nessus scan. Denne headeren beskytter mot mange ulike injiseringsangrep, sånn som clickjacking.

Jeg testet siden for clickjacking ved å kjøre en enkel test hvor jeg skrev en html-kode for en ny nettside og lastet Futura Buisness Informatique nettsiden inn i en frame på siden min. Deretter åpnet jeg siden i en nettleser, hvor jeg så at Futura Buisness Informatique var lastet opp og fungerte akkurat som på deres egen side.

Konsekvenser

Clickjacking er hovedsakelig et klient-side angrep. Det vil si at angrepet går utover klienten. Men siden det er hovedsakelig ansatte og samarbeidsparter som bruker siden, kan clickjacking brukes for å hente ut sensitiv informasjon som gir videre tilgang til serveren eller direkte tilgang til å transkribere penger ut av banken.

Konsekvenser for dere som en bank kan være utnyttning av viktige bank-funksjoner på siden deres. For eksempel muligheten til å utføre transaksjoner og utbetalinger. Her kan angriper legge på et element over et annet element som bruker ikke vil legge merke til engang, men som kan bety at man bekrefter transaksjoner man ikke vet om.

Angriper kan legge til elementer på siden som bruker av siden uvitende trykker på eller fyller ut. Disse elementene kan brukes til å hente ut sensitiv informasjon om pålogging, kontoer, transaksjoner, ansatte, personinformasjon, kredittinformasjon osv. Denne informasjonen kan brukes til for eksempel utpressing, komprimering av kontoer, datalekkasje, identitetstyveri og mye mer.

I tillegg til at dette er veldig alvorlige konsekvenser, vil nettsiden og banken også muligens bli oppfattet som utrygg, miste sin pålitelighet og få negativ omtale hos kunder og samarbeidsparter.

Løsning

Løsningen vil være å legge til headers som beskytter siden mot clickjacking – både X-Frame-Options header og CSP header.

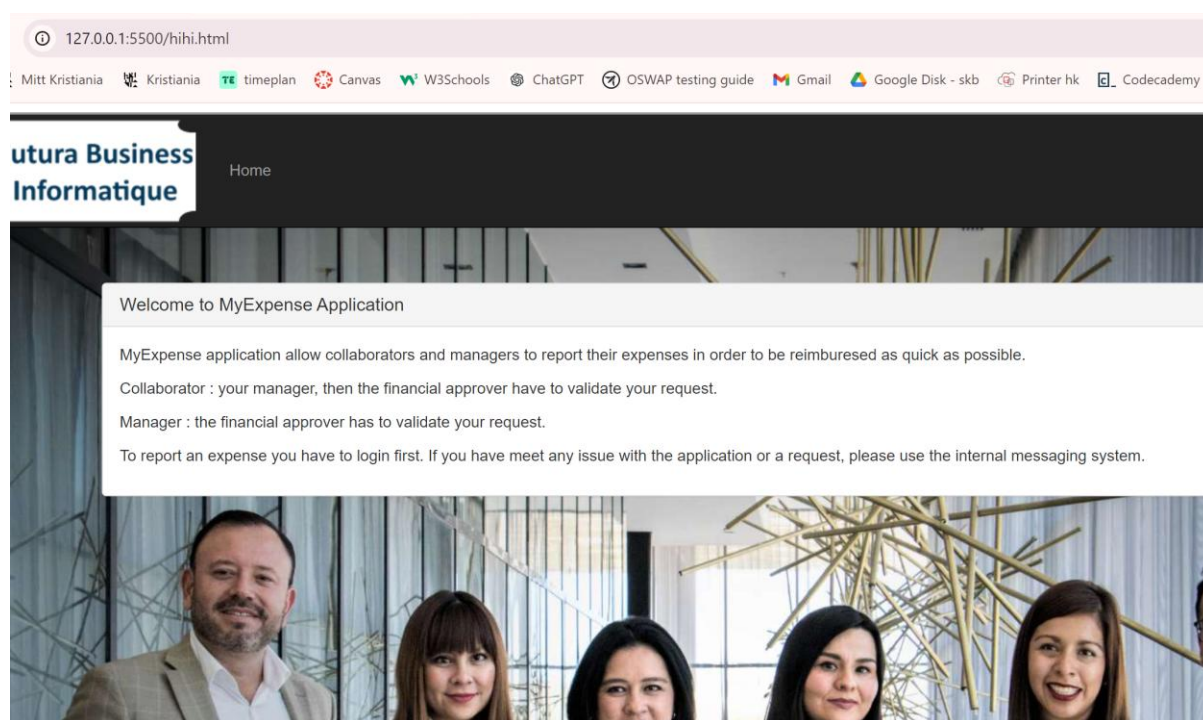
X-Frame-Options headeren beskytter siden mot clickjacking attacks ved å indikere om siden kan lastes inn i en frames eller objekter i andre siders script. Man kan velge hvorvidt man vil at siden skal kunne lastes opp i frames. Den kan settes til 3 verdier: DENY, SAMEORIGIN og ALLOW-FROM uri. DENY forhindrer at nettsiden kan lastes opp i frames i det hele tatt. SAMEORIGIN tillater opplastning i frames, men bare hos nettsider som har samme domene. ALLOW-FROM uri tillater opplastning i frames på sider med godkjente domener, denne brukes sjeldent lenger, ettersom at CSP headeren er et bedre alternativ. X-Frame-Options headeren legges direkte inn i kildekoden deres.

CSP headeren beskytter også mot clickjacking og jeg anbefaler også å ta i bruk denne. X-Frame-Options holder på å bli utdatert og erstattet med frame-ancestors i CSP headeren. Vurder hvilken dere helst vil bruke.

Dokumentasjon

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <iframe src="http://192.168.23.132" width="1500" height="1000"></iframe>
</body>
</html>
```

Figur 2.1: HTML kode som lager en egen side, hvor FBI lastes inn i en frame



Figur 2.2: HTML siden over, som viser FBI suksessfullt lastet opp i siden

Sårbarhet 3 - SQLi

Sårbar for SQL injection. Serveren kan være sårbar for sql injection ettersom nettsiden returnerer forskjellig ...

MÅ SJEKKES UT

Oppdagelse

Konsekvenser

Løsning

Dokumentasjon

Sårbarhet 4 – Apache versjon 2.4.38

Apache versjonen som brukes er utdatert. Serveren bruker apache versjon 2.4.38 som har mange kjente sårbarheter. Versjonen er fra 2018 og siden det har det kommet mange oppdateringer som er mye tryggere å bruke. Å kjøre en utdatert versjon er ikke ideelt ettersom at senere versjoner har oppdatert og fikset kjente sikkerhetsfeil og sårbarheter.

Her er 3 av de mest kjente og kritiske sårbarhetene ved Apache versjon 2.4.38 som alle har risiko nivå 9.8 (kritisk):

CVE-2021-44790

Denne sårbarheten kan føre til en buffer overflow i `mod_lua`, hvor en spesiallaget forespørsel kan overskrive minnet. Selv om det ikke finnes kjente utnyttelser for dette, kan en angriper potensielt bruke det til å krasje serveren eller kjøre ondsinnet kode.

CVE-2021-39275

Denne sårbarheten kan føre til at funksjonen `ap_escape_quotes()` skriver data utenfor den tildelte bufferen dersom den får ondsinnet input. Selv om ingen innebygde moduler sender ukontrollert data til denne funksjonen, kan eksterne eller tredjepartsmoduler gjøre det, noe som potensielt kan utnyttes av angripere.

CVE-2021-26691

Denne sårbarheten innebærer at en spesiallaget SessionHeader sendt fra en opprinnelseserver kan føre til heap overflow. Dette betyr at ondsinnet data kan overskrive deler av minnet (heap) som vanligvis brukes til dynamisk minnetildeling. Dette kan føre til krasj eller mulig utføring av vilkårlig kode, noe som gjør systemet sårbart for angrep.

Dette er bare noen av sårbarhetene som finnes ved versjon 2.4.38, for en mer utfyllende og detaljert beskrivelse av alle sårbarhetene ville jeg sjekket ut denne siden:

https://httpd.apache.org/security/vulnerabilities_24.html

Oppdagelse

Fikk opp versjonsnummer for apache i både nikto scan og owasp zap. I tillegg for å sjekke at dette stemte logget jeg meg inn på serveren og skrev `apt-cache policy apache2` for å få apache versjonen som kjører på serveren. Da fikk jeg opp samme versjon som jeg hadde fått i scannene. Versjon 2.4.38

Konsekvenser

Konsekvensene av å bruke en utdatert apache versjon vil være at en angriper kan utnytte de kjente sårbarhetene til å angripe serveren.

Løsning

Løsningen vil være å oppdatere til en nyere versjon som har løst mange av sikkerhetsfeilene.

Dokumentasjon

```
osboxes@osboxes:~$ apt-cache policy apache2
apache2:
  Installed: 2.4.38-3+deb10u6
  Candidate: 2.4.38-3+deb10u6
  Version table:
*** 2.4.38-3+deb10u6 500
    500 http://security.debian.org/debian-security buster/updates/main amd64
Packages
  100 /var/lib/dpkg/status
  2.4.38-3+deb10u5 500
    500 http://httpredir.debian.org/debian buster/main amd64 Packages
```

Figur 4.1: kommando i terminal til serveren, som forteller hvilken apache versjon som kjører

Sårbarhet 5 – tilgang til admin.php

Tilgang til admin/admin.php er ikke blokkert og returnerer http 200. Denne siden er en admin side som er viser konfidensiell informasjon om brukernavn, navn, etternavn, mailadresse, rolle, aktivitet og status på brukerne. Siden er lett tilgjengelig for uvedkommende, dersom de skulle finne den.

Oppdagelse

Dette ble oppdaget gjennom en nikto scan av IP adressen og ved en rask sjekk om siden var tilgjengelig ved å skrive inn siden jeg fikk av nikto scan i adressefeltet. I tillegg sjekket jeg owasp zap hva siden returnerte og fikk opp at den returnerte http 200 OK. (spider crawl og)

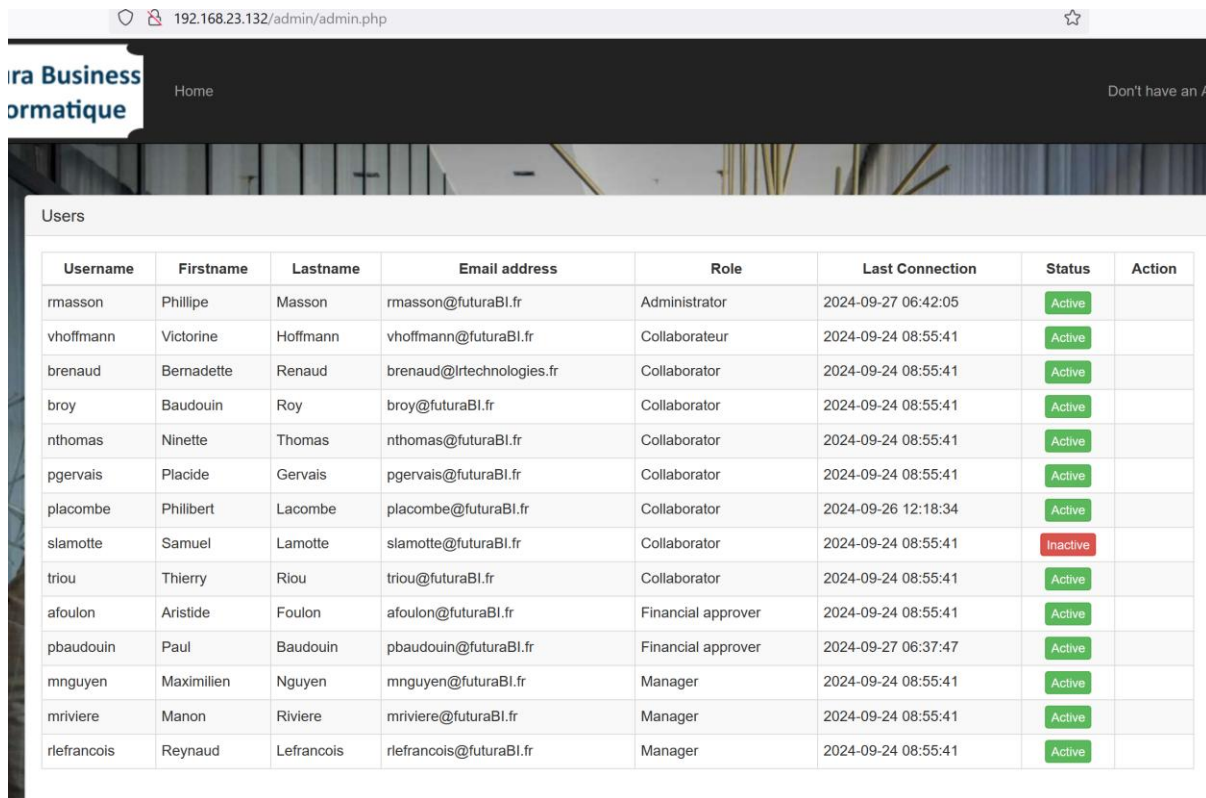
Konsekvenser

En eventuell innbryter kan bruke denne informasjonen til å prøve å logge seg inn, knekke passord, social engineering, phishing og andre angrep som involverer å bruke brukeren. Dette gjør det betydelig enklere å hacke siden.

Løsning

Her anbefaler jeg å fjerne siden fra robot.txt og å sette opp tilgangskontroller som sjekker om personen som prøver å komme seg til siden, faktisk har autorisasjon og er admin. I tillegg ville jeg konfigurert serveren til å returnere 403 forbidden i stedet for 200 ok for admin siden.

Dokumentasjon



The screenshot shows a web browser window with the address bar displaying '192.168.23.132/admin/admin.php'. The page title is 'Futura Business Informatique'. The main content area is titled 'Users' and contains a table with the following data:

Username	Firstname	Lastname	Email address	Role	Last Connection	Status	Action
rmasson	Phillipe	Masson	rmasson@futuraBI.fr	Administrator	2024-09-27 06:42:05	Active	
vhoffmann	Victorine	Hoffmann	vhoffmann@futuraBI.fr	Collaborateur	2024-09-24 08:55:41	Active	
brenaud	Bernadette	Renaud	brenaud@lrtechnologies.fr	Collaborator	2024-09-24 08:55:41	Active	
broy	Baudouin	Roy	broy@futuraBI.fr	Collaborator	2024-09-24 08:55:41	Active	
nthomas	Ninette	Thomas	nthomas@futuraBI.fr	Collaborator	2024-09-24 08:55:41	Active	
pgervais	Placide	Gervais	pgervais@futuraBI.fr	Collaborator	2024-09-24 08:55:41	Active	
placombe	Philibert	Lacombe	placombe@futuraBI.fr	Collaborator	2024-09-26 12:18:34	Active	
slamotte	Samuel	Lamotte	slamotte@futuraBI.fr	Collaborator	2024-09-24 08:55:41	Inactive	
triau	Thierry	Riou	triau@futuraBI.fr	Collaborator	2024-09-24 08:55:41	Active	
afoulon	Aristide	Foulon	afoulon@futuraBI.fr	Financial approver	2024-09-24 08:55:41	Active	
pbaudouin	Paul	Baudouin	pbaudouin@futuraBI.fr	Financial approver	2024-09-27 06:37:47	Active	
mnguyen	Maximilien	Nguyen	mnguyen@futuraBI.fr	Manager	2024-09-24 08:55:41	Active	
mriviere	Manon	Riviere	mriviere@futuraBI.fr	Manager	2024-09-24 08:55:41	Active	
rlfrancois	Reynaud	Lefrancois	rlfrancois@futuraBI.fr	Manager	2024-09-24 08:55:41	Active	

Figur 5.1: bilde av admin/admin.php siden som er tilgjengelig

GET	http://192.168.23.132/admin/admin.php	200 OK
-----	---------------------------------------	--------

Figur 5.2: owasp zap som viser at siden returnerer 200 OK

Sårbarhet 6 - XSS

Siden mangler Content Security Policy response header og er sårbar for XSS angrep. CSP headeren begrenser hvilke script og filer som kan brukes på siden. XSS angrep er

Oppdagelse

I en owasp zap scan fikk jeg opp at Content Security Policy header ikke var satt. Dette er en parameter som beskytter siden mot ulike injeksjonsangrep som blant annet XSS.

I en nikto scan og owasp zap fikk jeg også opp noen andre sårbarheter som ikke direkte tillater XSS men som kan videre føre til XSS angrep. For eksempel at cookies ikke har SameSite Attribute eller httponly flagg og at X-content-Type-Options header ikke er til stede.

Fikk også opp i Nessus at CSP headeren ikke er til stede.

Alle disse funnene indikerer at siden er sårbar for XSS angrep, derfor ville jeg teste om den faktisk var det. Dette gjorde jeg ved å teste de ulike input feltene med javascript kode. Da jeg gjorde dette i meldingsfeltet på admin-siden, ble koden kjørt.

Konsekvenser

Cross Site Scripting hvor angiper kan skrive inn javascript kode i et input felt for å manipulere siden. Dette kan føre til at angriper kan få tilgang til sensitiv informasjon, ta over brukeren, injisere virus osv. Det finnes mange konsekvenser for disse typer angrep, siden angriper kan skrive nesten hva slags kode han vil.

Løsning

Løsningen vil være å sanitere input som sendes. Dette vil gjøres ved å sette htmlspecialchars i koden for input. I tillegg anbefaler jeg på det stekeste å sette CSP header. For å bedre sikre mot XSS angrep ville jeg satt SameSite attribute og httpOnly flagg på cookies og sette en X-Content-Type-Options header. (hva gjør disse? Hvorfor beskytter de mot xss? Lurt å skrive etter du har jobbet med de sårbarhetene)

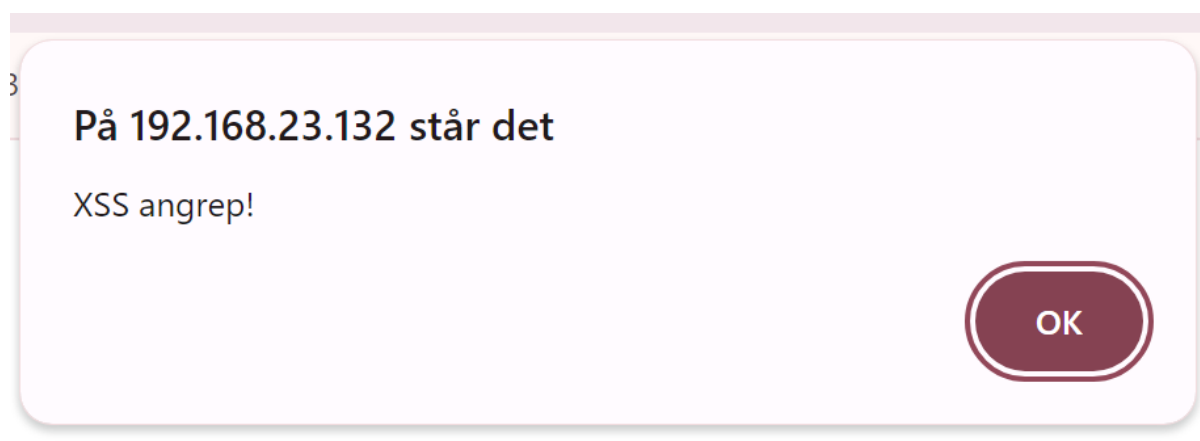
Dokumentasjon

Your message :

```
<script>alert("XSS angrep!")</script>
```

Post your message

Figur 6.1: javascript kode skrevet inn i meldingsfelt inne på admin bruker



Figur 6.2: javascript koden skrevet inn i meldingsfelt som kjører og viser en alert

Sårbarhet 7 – X-Content-Type-Options og MIME-sniffing

Manglende X-Content-Type-Options header. Dette er en header som beskytter siden mot MIME-sniffing ved å si at innholdet skal være den MIME typen som er spesifisert i content-type forblir det den er satt til og ikke endres. Denne headeren angir at MIME-typene er bevisst konfigurert. MIME-typer hjelper servere med å fortelle nettlesere hva slags data en fil inneholder og hvordan de skal behandle de. Dette kan være fks html dokumenter, jpeg bilder og JSON data.

MIME-sniffing er en prosess hvor nettleseren ignorerer MIME-typen i content-type for en fil og prøver å gjette hva den inneholder basert på innholdet i seg selv. Det er ikke alltid at nettleseren gjetter riktig og dette kan føre til at brukeren er sårbar for flere typer angrep.

X-Content-Type-Options forteller nettleseren at den alltid skal følge MIME-typen spesifisert i Content-typen og forhindrer MIME-sniffing.

Oppdagelse

Denne sårbarheten ble oppdaget i nikto scan, owasp zap og nessus.

Konsekvenser

MIME-sniffing kan føre til XSS angrep hvor filer som ikke er kode, tolkes som kode, slik at en angriper kan injisere ondsinnet kode i en brukers nettleser. (Se konsekvenser for XSS, sårbarhet 6).

Løsning

Sett en X-Content-Type-Options header til nosniff.

Sårbarhet 8 – Cookies uten httpOnly flag

Cookie uten httpOnly flag. HttpOnly flagget gjør at cookiene ikke kan endres med javascript.

Oppdagelse

Denne sårbarheten ble oppdaget med nikto scan.

Konsekvenser

Konsekvensene av denne sårbarheten kan være XSS angrep. Cookiene kan bli tilgjengelige for en angriper gjennom injisering av javascript kode, slik at angriper kan lese cookiene til en bruker.

Dette kan igjen føre til session hijacking som er at en angriper får tilgang til sesjons ID slik at han kan logge seg inn på brukeren.

Angriper kan også manipulere cookiene.

Løsning

Løsningen vil være å legge til en httpOnly flag til cookiene.

Sårbarhet 9 – Cookies uten sameSite attribute

Cookie uten sameSite attribute. sameSite Attribute passer på at cookiene som blir sendt kommer fra samme side.

Oppdagelse

Dette ble oppdaget ved en owasp zap scan

Konsekvenser

Cross-Site Request Forgery som vil si at cookiene kan bli sendt fra en ekstern nettside.

Løsning

Bruke sameSite attribute på cookiene.

Sårbarhet 10 – databasefil ikke slettet

Fil for å opprette databasen er ikke slettet og inneholder sensitiv informasjon om passordene til alle brukerne i klartekst.

Oppdagelse

Ved gjennomgang av kildekode, kom jeg over en fil som inneholder kode for å opprette databasen til serveren deres.

Konsekvenser

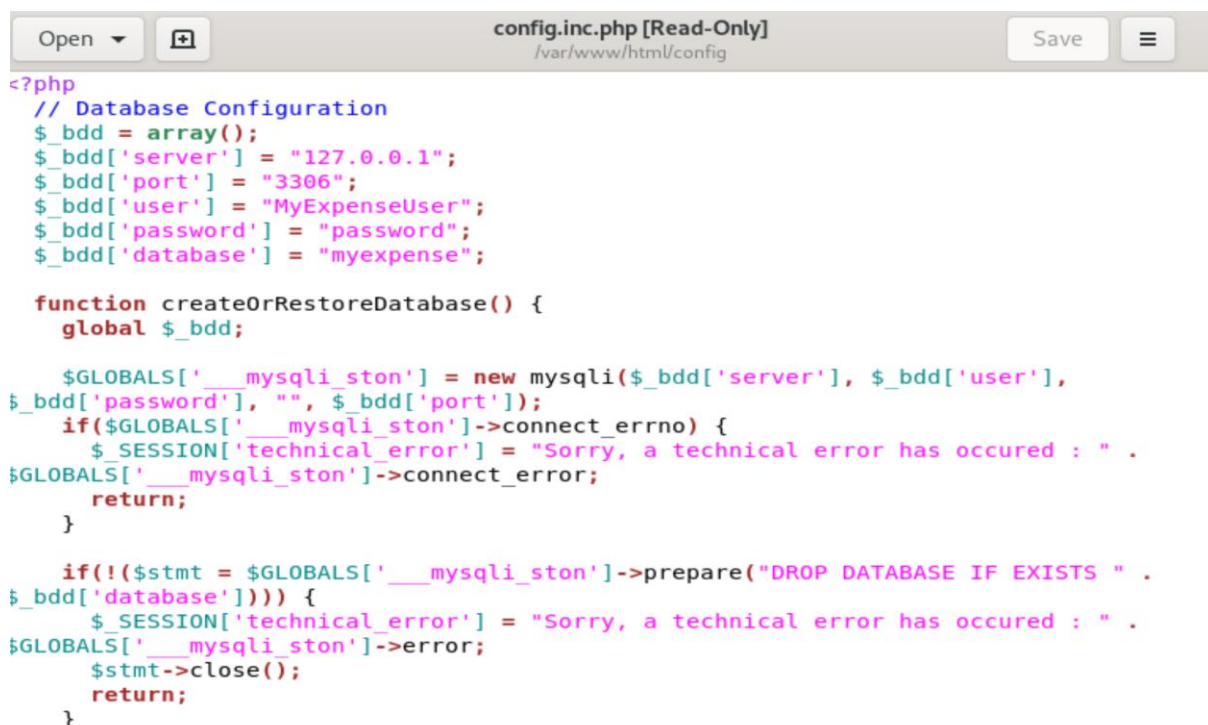
Konsekvensen av dette er at hvis noen noen gang skulle fått tilgang til serveren deres, vil alt som er lagret i databasen være tilgjengelig i dokumentet. Her står det sensitiv informasjon i klartekst, slik som alle passordene på alle brukere.

Løsning

Denne filen skulle vært sletta etter opprettelsen av databasen. Løsningen vil være å slette den så fort som mulig.

Dokumentasjon

Her ser man filen åpnet og at den inneholder konfigurasjonen av databasen og passord i klartekst:



```
<?php
// Database Configuration
$_bdd = array();
$_bdd['server'] = "127.0.0.1";
$_bdd['port'] = "3306";
$_bdd['user'] = "MyExpenseUser";
$_bdd['password'] = "password";
$_bdd['database'] = "myexpense";

function createOrRestoreDatabase() {
    global $_bdd;

    $GLOBALS['__mysqli_ston'] = new mysqli($_bdd['server'], $_bdd['user'],
$_bdd['password'], "", $_bdd['port']);
    if($GLOBALS['__mysqli_ston']->connect_errno) {
        $_SESSION['technical_error'] = "Sorry, a technical error has occurred : " .
$GLOBALS['__mysqli_ston']->connect_error;
        return;
    }

    if(!($stmt = $GLOBALS['__mysqli_ston']->prepare("DROP DATABASE IF EXISTS " .
$_bdd['database']))) {
        $_SESSION['technical_error'] = "Sorry, a technical error has occurred : " .
$GLOBALS['__mysqli_ston']->error;
        $stmt->close();
        return;
    }
}
```

Figur 10.1: konfigurasjonsfil til databasen åpnet

```
if(!($stmt = $GLOBALS['__mysqli_ston']->prepare("INSERT INTO user VALUES
(1, 'afoulon', '' . md5('wq6hblv3') . '', 'Financial approver', 'Foulon',
'Aristide', 1, 'afoulon@futuraBI.fr', 1, NOW(), 1),
(2, 'pbaudouin', '' . md5('HackMe') . '', 'Financial approver', 'Baudouin',
'Paul', 1, 'pbaudouin@futuraBI.fr', 2, NOW(), 1),
(3, 'rlefrancois', '' . md5('m4ukhkjq') . '', 'Manager', 'Lefrancois', 'Reynaud',
'rlefrancois@futuraBI.fr', 1, NOW(), 1),
(4, 'mriviere', '' . md5('6v7j4tj2') . '', 'Manager', 'Riviere', 'Manon', 2,
'riviere@futuraBI.fr', 2, NOW(), 1),
(5, 'mnguyen', '' . md5('27sfbdi') . '', 'Manager', 'Nguyen', 'Maximilien', 3,
'mnguyen@futuraBI.fr', 2, NOW(), 1),
(6, 'pgervais', '' . md5('b98e67ys') . '', 'Collaborator', 'Gervais', 'Placide',
'pgervais@futuraBI.fr', 3, NOW(), 1),
(7, 'placombe', '' . md5('qayhned2') . '', 'Collaborator', 'Lacombe',
```

Figur 10.2: lenger ned i filen, som viser passordene deres i klartekst

Sårbarhet 11 – md5 kryptering

Passord er kryptert med hashing md5. md5 er en utdattert og usikker hashing algoritme. Den er mye enklere å dekryptere enn andre mer moderne hashing algorithmer. Dette er fordi md5 baserer seg på 128 bit i stedet for 256 bit som mange andre hashing algoritmer bruker.

Oppdagelse

Jeg kom over dette ved en gjennomgang av kildekoden, da jeg så gjennom filen config.inc.php som er filen for konfigurering av databasen.

Konsekvenser

Md5 er sårbar for collision attacks som er et angrep der to forskjellige innhold som krypteres, i dette tilfelle passord kan generere samme hash-verdi. Dette kan utnyttes av angripere ved at de kan lage ett annet passord som gir samme hash og deretter bruke denne i systemet som stoler på md5 for passordverifisering, i stedet for passordet selv.

I tillegg er md5 mye enklere å beregne enn andre krypteringer.

Løsning

Løsningen vil være å kryptere passordene med en sterkere hash.

Dokumentasjon

```
if(!($stmt = $GLOBALS['__mysqli_ston']->prepare("INSERT INTO user VALUES
(1, 'afoulon', '' . md5('wq6hblv3') . '', 'Financial approver', 'Foulon',
'Aristide', 1, 'afoulon@futuraBI.fr', 1, NOW(), 1),
(2, 'pbaudouin', '' . md5('HackMe') . '', 'Financial approver', 'Baudouin',
'Paul', 1, 'pbaudouin@futuraBI.fr', 2, NOW(), 1),
(3, 'rlefrancois', '' . md5('m4ukhkjq') . '', 'Manager', 'Lefrancois', 'Reynaud',
'rlefrancois@futuraBI.fr', 1, NOW(), 1),
(4, 'mriviere', '' . md5('6v7j4tj2') . '', 'Manager', 'Riviere', 'Manon', 2,
'mriviere@futuraBI.fr', 2, NOW(), 1),
(5, 'mnguyen', '' . md5('27sfbdi') . '', 'Manager', 'Nguyen', 'Maximilien', 3,
'mnguyen@futuraBI.fr', 2, NOW(), 1),
(6, 'pgervais', '' . md5('b98e67ys') . '', 'Collaborator', 'Gervais', 'Placide',
'pgervais@futuraBI.fr', 3, NOW(), 1),
(7, 'placombe', '' . md5('qayhned2') . '', 'Collaborator', 'Lacombe',
```

Figur 11.1: Fra databasefilen, hvor man ser at passordene er hashet med md5

Sårbarhet 12 – database passord

Svakt passord på databasen. I databasen brukes det et veldig svakt passord. Dette passordet finnes i mange passordlister som brukes for å knekke passord. Dette kan enkelt knekkes og gjør databasen sårbar.

Oppdagelse

Dette oppdaget jeg da jeg gikk gjennom filen for konfigurering av database. Config.inc.php.

Konsekvenser

Dette kan lett knekkes av passordlister, som vil føre til full tilgang til databasen.

Løsning

Løsningen er å endre passordet til et sterkere passord. Gjerne en setning som ikke relaterer til firmaet med tegn.

Dokumentasjon

```
<?php
// Database Configuration
$_bdd = array();
$_bdd['server'] = "127.0.0.1";
$_bdd['port'] = "3306";
$_bdd['user'] = "MyExpenseUser";
$_bdd['password'] = "password";
$_bdd['database'] = "myexpense";
```

Figur 12.1: Fra database konfig fil, hvor man ser at passordet er password

Informasjon 1 – README ikke blokkert

Apache /icons/README default fil er ikke blokkert.

Informasjon 2 – Falsk positiv

Falsk positiv – admin login page found

Informasjon 3

Informasjon 4

Informasjon 5

Informasjon 6

Informasjon 7

Kilder

https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/02-Fingerprint_Web_Server

owasp zap testing guide

https://httpd.apache.org/security/vulnerabilities_24.html

<https://vulmon.com/searchpage?q=apache+http+server+2.4.38&sortby=byriskscore>

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options>

<https://www.liquidweb.com/blog/how-to-check-the-version-of-apache/>

OSV